COMP 403

Parallel and distributed processing

Lecture 2

28-9-2026

Outline

- Revision of
  - Properties of Concurrent Programs.
  - The Critical Section Problem
    - Semaphores.
- The Producer and Consumer Problem
  - Using unbounded buffer
- The Dining Philosophers Problem

Safety and Liveness Properties

Safety Property

- **Mutual exclusion** — at any time, no particular resource is used by more than one process.
- **No deadlocks** — a situation in which two or more processes cannot terminate because they cannot continue their actions.

Liveness Property

- During each concurrent implementation of a program, a required condition will be satisfied at some point.
- There will be no individual starvation of a process because the resource is not allocated to it.

The Critical Section Problem

- Occurs when a group of processes compete for the resource, where at any given time only one process can have access to it.
- Resources of this type:
  - variables in shared memory
  - database records
  - files
  - physical devices (printers, …)
- Critical section:
  - The fragment of a process (or a task) in which it makes use of a resource
- Ensuring exclusive: At any time, only one task executes its critical section.
- The critical section problem can be solved employing semaphores.

Semaphores

- Semaphore *s* is a compound data structure with two fields:
  - *s.w* : nonnegative integer values,
  - *s.q* : sets of tasks (processes).
- Examples:
  - s: semaphore := (1, ϕ)
- Two types of semaphores:
  1. **General semaphore**: s.w is any nonnegative integer value
  2. **Binary semaphore**: s.w is only 0 or 1 (1 : free , 0 : used by others)

```
type semaphore is
    record
        w: natural;
        q: queue;
    end record;
```

Semaphores Operations

wait(*s*): If $s.w > 0$, then $s.w := s.w - 1$,
else suspend execution of the task performing operation *wait* and add it to set $s.q$.
The task added to set $s.q$ is said to be blocked on semaphore *s*.

signal(*s*): If set $s.q$ of tasks is nonempty,
then delete and unblock one of the tasks from $s.q$ and resume its execution,
else $s.w := s.w + 1$.

Solving the Producer and Consumer Problem with an unbounded buffer

```
buffer: array(0 ..∞) of data_type; -- unbounded buffer
  in, out: natural := 0;  -- buffer indices
  nonempty: semaphore := (0, ∅);
-- general semaphore indicating how many chunks of data are in the buffer
 -- initially there is no data in the buffer
```

Solving the Producer and Consumer Problem with an unbounded buffer (cont.)

```
task Producer;
task body Producer is
   data: data_type; – – auxiliary variable containing
                    --a  chunk of data
 begin
    loop
       Produce data;
      buffer(in) := data; – – insert data into the
buffer
      in := in + 1;
     signal (nonempty); – – signal data insertion
end loop;
 end Producer;
```

```
task Consumer;
task body Consumer is
data: data _type; – – auxiliary variable
 begin
  loop
    wait(nonempty); – – check if there are data in the
buffer
    data := buffer(out); – – retrieve data from the
buffer
    out := out + 1;
    Consume data;
 end loop;
end Consumer;
```

Condition Synchronization with *nonempty*

- The semaphore *nonempty* indicates whether the producer has inserted data into the buffer.
- Depending on the buffer state:
  - nonempty.w > 0 → data is available
  - nonempty.w = 0 → buffer is empty
- This use of semaphore is refereed to as **condition** (or **event**) synchronization.

Condition synchronization is essential when:

- A task can perform an operation correctly **only if** another task has already executed its operation, **or**
- A **specific event has occurred**.

Problem with unbounded buffer

Memory exhaustion

- In practice, unbounded buffers are theoretical — real buffers have finite size.
- If the producer is much faster than the consumer, the buffer fills up and memory can overflow.

Solving the Producer and Consumer Problem with a bounded buffer

buffer: **array**(0 ..n-1) **of** *data_type*; -- bounded buffer of n elements
in, *out*: *natural* := 0; -- buffer indices
nonempty: *semaphore* := (0, ∅);
-- general semaphore indicating how many chunks of data are in the buffer
nonfull : *semaphore* := (n, ∅);
-- general semaphore indicating **how many free entries** are in the buffer
--nonfull is used in order to prevent inserting data into full buffer

Buffer is now an *n*-element array of cyclic organization:

- When a chunk of data is inserted to the buffer at position $n - 1$, the value of index *in* designating the first free entry in the buffer is set to 0.
- The *out* index is modified in a similar way after retrieving data from position $n - 1$.

```
in:= (in +1) mod n; out := (out +1) mod n;
```

Solving the Producer and Consumer Problem with a bounded buffer (cont.)

```
task Producer;
task body Producer is
   data: data_type; – – auxiliary variable containing a
                       -- chunk of data
 begin
      loop
       Produce data;
      wait(nonfull); -- check if the buffer is not full
      buffer(in) := data; – – insert data into the buffer
     in := (in + 1) mod n;
     signal (nonempty); – – signal data insertion
end loop;
 end Producer;
```

```
task Consumer;
task body Consumer is
data: data _type; – – auxiliary variable
  begin
     loop
   wait(nonempty); – – check if there are data in the buffer
   data := buffer(out); – – retrieve data from the buffer
    out := (out + 1) mod n;
   signal(nonfull); -- signal that the buffer is not full
    Consume data;
 end loop;
end Consumer;
```

Solving the Producer and Consumer Problem with a bounded buffer (cont.)

```
task Producer;
task body Producer is
   data: data_type; – – auxiliary variable cont
                      -- chunk of data
 begin
      loop
       Produce data;
      wait(nonfull); -- check if the b
      buffer(in) := data; – – insert da
     in := (in + 1) mod n;
    signal (nonempty); – – signal data insertion
end loop;
 end Producer;
```

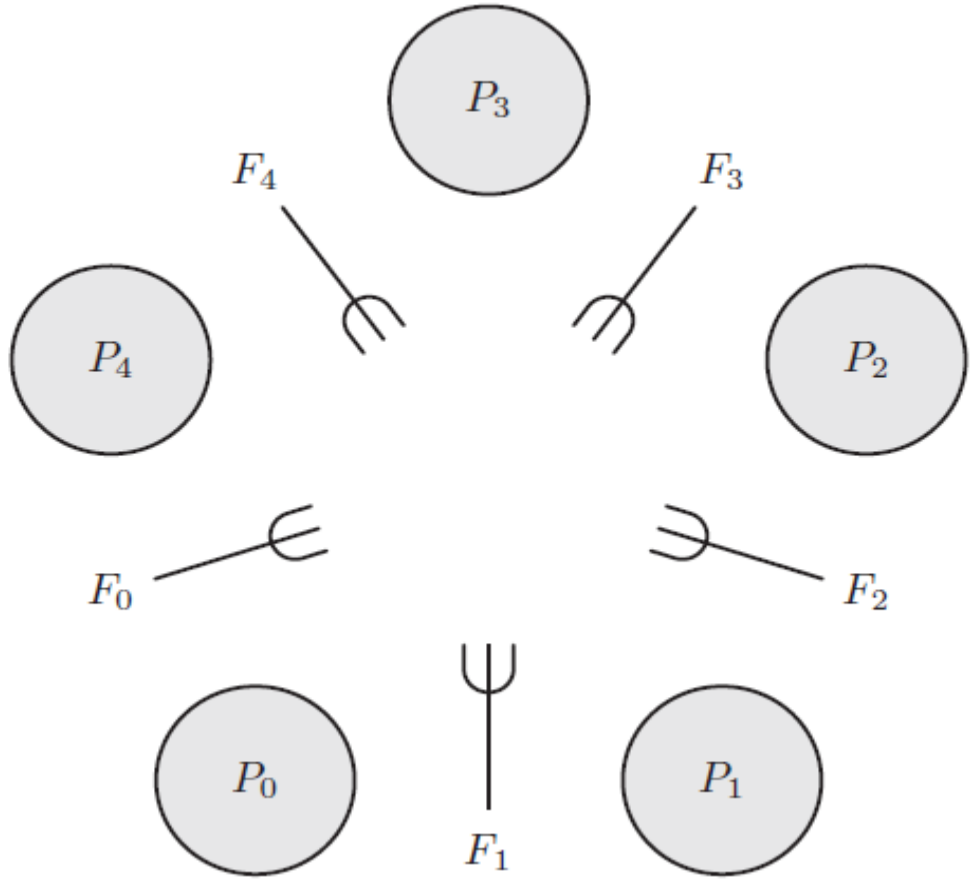
Protects against retrieving data from the empty buffer

Protects insertions data into full buffer

```
task Consumer;
task body Consumer is
   data _type; – – auxiliary variable
   n
   op
    wait(nonempty); – – check if there are data in the buffer
    data := buffer(out); – – retrieve data from the buffer
    out := (out + 1) mod n;
    signal(nonfull); -- signal that the buffer is not full
     Consume data;
 end loop;
end Consumer;
```

The Dining Philosopher Problem

- Five philosophers sit at a round table, spending time thinking and eating.
- At the center: a bowl of spaghetti, constantly replenished.
- Five plates — one in front of each philosopher.
- A philosopher needs two forks to eat.
- The forks lie on the table, one to the left and one to the right of each philosopher.



The Dining Philosopher Problem (cont.)

- There are only five forks, and each can be used by only one philosopher at a time.
- At most two philosophers not sitting next to each other can eat simultaneously.
- Philosophers can be viewed as tasks that require access to two resources **(**forks**)** to carry out their activity (eating).

The problem:

- Properly synchronize access to the forks, allowing each philosopher to alternate between eating and thinking.

The Dining Philosopher Problem (cont.)

- Assume the philosophers are numbered from 0 to 4, and that philosopher *i* has fork *i* on their left and fork (*i* + *1*) *mod 5* on their right.
- Forks are represented by binary semaphores:

```
fork: array(0 .. 4) of semaphore := (0 .. 4 => (1, ∅));
```

The Dining Philosopher Problem: First Attempt

```
1 task Philosopher_i;                  – – for i = 0, 1, . . . , 4
2 task body Philosopher_i is
3 begin
4     loop
5         Think;
6         wait(fork(i));          – – pick up the left fork
7         wait(fork((i + 1) mod 5));      – – pick up the right fork
8         Eat;
9         signal (fork(i));               – – put down the left fork
10        signal (fork((i + 1) mod 5));   – – put down the right fork
11    end loop;
12 end Philosopher_i;
```

The Dining Philosopher Problem: First Attempt (cont.)

- Each philosopher is represented by a single task.

Behavior:

- Pick up the left fork, then the right fork (lines 6–7).
- If a fork is unavailable, the philosopher must wait.
- Once both forks are acquired → eat, then release the forks (lines 9–10).

Problem: This solution can lead to deadlock and starvation.

How deadlock occurs:

- All philosophers pick up the left fork simultaneously (line 6).
- Each then waits indefinitely for the right fork (line 7).

The Dining Philosopher Problem: Deadlock Free Solution

- Reduce the number of philosophers to four.
- This ensures that at least one philosopher will be able to access both forks.

The Dining Philosopher Problem: Deadlock Free Solution (cont.)

```
1 up_to_4: semaphore := (4, ∅);  – – general semaphore limiting the number of philosophers at the table to 4
2 task Philosopher_i;            – –  for i = 0, 1, . . . , 4
3 task body Philosopher_i is
4 begin
5  loop
6     Think;
7     wait(up_to_4);             – – get permission to have a seat at the table
8     wait(fork(i));             – – pick up the left fork
9     wait(fork((i + 1) mod 5)); – – pick up the right fork
10        Eat;
11 signal (fork(i)); – – put down the left fork
12 signal (fork((i + 1) mod 5)); – – put down the right fork
13 signal (up_to_4); – – release the seat at the table
14 end loop;
15 end Philosopher_i;
```

Correctness of the Solutions

1- Mutual exclusion on forks:

- No fork is used by more than one philosopher at a time — access is managed via a binary semaphore, so each fork is allocated to only one philosopher.

Correctness of the Solutions (cont.)

2-Starvation cannot occur — proof by contradiction:

1. suppose philosopher i is blocked forever on semaphore **up_to_4.**
2. That means **4 philosophers** are already at the table — otherwise i would have been admitted.
3. At least one of them can **pick up 2 forks**; after eating, that philosopher **leaves the table** by performing signal(up_to_4)
4. Philosopher i then **gains access** — **contradicting** the assumption.
5. Therefore, **starvation cannot occur**.

Question 1

- Consider the following binary semaphores and tasks:

  a: *semaphore* := (1, ∅);

  b: *semaphore* := (0, ∅);

```
task A;
task body A is
    wait(a);
    write ('A');
     signal(a);
end A;
```

```
task B;
task body B is
    wait(a);
    write ('B');
     signal(b);
     signal(a);
end B;
```

```
task C;
task body C is
    wait(b); wait(a);
    write ('C');
     signal(a);
end C;
```

- Specify the possible output messages that can be printed as the result of concurrent implementation of tasks *A*, *B and C*

Question 1 (cont.)

Possible execution sequences:

- A runs first:

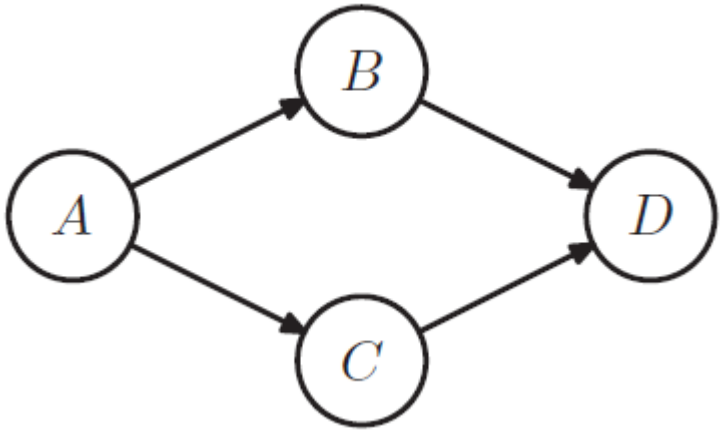
  ABC

- B runs first:

  BAC, BCA

Question 2

- Given the tasks *A*, *B*,*C*, and *D* synchronize their execution by using semaphores according to the dependency graph:



- The first task to execute is *A*, then tasks *B* and *C* should be executed in parallel, and finally task *D*.

Question 2 (cont.)

S_B : semaphore := (0, ∅);

S_C : semaphore := (0, ∅);

done_B : semaphore := (0, ∅);

done_C : semaphore := (0, ∅);

Question 2 (cont.)

```
task A;
task body A is
   Operations A;
   signal(S_B);
   signal(S_C);
end A;
```

```
task B;
task body B is
       wait(S_B);
       Operations B;
      signal(done_B);
end B;
```

```
task C;
task body C is
       wait(S_C);
       Operations C;
       signal(done_C);
end C;
```

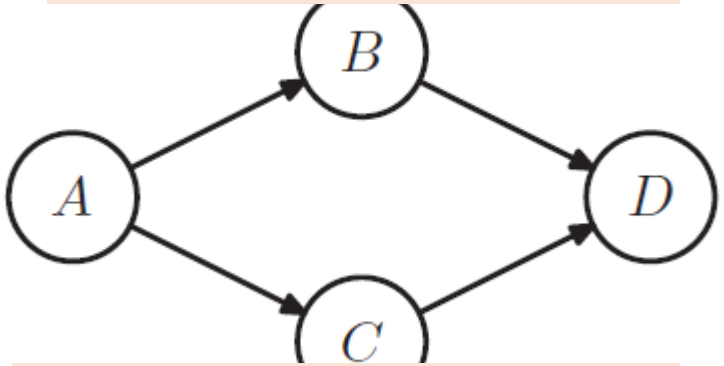
```
task  D;
task body D is
       wait(done_B);
     wait(done_C);
       Operations D;
end D;
```

fork-join Parallelism Pattern

A fundamental synchronization model in concurrent programming:

- Fork: A single task splits into multiple (two or more) parallel subtasks that execute concurrently
- Join: The parallel subtasks eventually synchronize back to a single point of execution

Fork (B and C run in parallel)



Join (D waits for both B and C)

Single Entry Point: One task (A) starts the process
Parallel Execution: Multiple tasks (B, C) run simultaneously after the fork
Synchronization Point: One task (D) waits for all parallel tasks to complete before proceeding
Barrier-like Behavior: The "join" acts as a barrier that all parallel tasks must reach